

# Testing

# Contents

- What is Testing
- Static and Dynamic Testing
- Test Techniques and Approaches
- Test Levels
- Test Types
- Testing Change
- Test Planning
- Test Tools
- Defect Management
- CW Test Case Example

# What is Testing



# What is Testing

- Software testing is an assessment of software quality
- Testing helps to verify and validate the software system under development
  - It reduces the risk of software failure in operation
  - It can also help to ensure software meets stakeholder needs
- To achieve this, testing must be broader than simply executing tests against code
- It is a complex intellectual activity requiring careful planning and management
- As such, testers require specialised knowledge, and must be analytical, critical, thinkers
- Increasingly, testing is proactive rather than passive

# Objectives of Testing

- Evaluating work products, e.g. requirements, user stories, designs, and code
- Finding code defects
- Ensuring full test coverage, so that nothing is missed, as far as possible
- Reducing the risk present within a software system
- Verifying that specified requirements have been fulfilled
- Verifying the software meets contractual, legal, and regulatory requirements
- Validating that the software is complete and works as stakeholders expected
- Providing information to inform stakeholder decisions
  
- Note, test objectives vary depending upon the context
  - for example: SDLC being used, project stage, and test level

# Testing Principles

- The principles of testing can help to understand what testing is, and what it is not
- Testing reduces the probability of defects but cannot prove the absence of defects
- Testing cannot be exhaustive and hence requires some form of prioritisation
- Testing early reduces derived defects and the time and cost required to fix them
- Defects tend to be clustered in keeping with the Pareto Principle (80/20 Rule)
- Tests and test data must be reviewed and maintained to ensure they remain useful
- Testing depends on context, there is no one-size fits all
- Testing will reduce defects but it will not ensure the success of the resulting system



# High-level Test Activities

- Testing can be grouped into a number of activities
  - Note, the order, importance and any repetition depends on the overarching SDLC
- **Planning**, defining the test objectives and selecting an approach that suits the objectives
- **Monitoring and control**, monitoring and controlling test activities against the objectives
- **Test analysis**, identifying and prioritising features, and evaluating testability, based on risk
- **Test design**, defining test cases, test data, test environment, and identifying tools needed
- **Test implementation**, creating, acquiring and organising test data, scripts, environment
- **Test execution**, run tests, compare test/expected results, log results, analysed anomalies
- **Test completion**, record defects, review test activities, and report outcomes
- As you can see, much of this is not simply about running tests on code

# Static and Dynamic Testing



# Static Testing

- Static testing is a class of testing that does not require code execution
- Indeed, it is not limited to testing code, and can be used to test a range of work products
  - Can be used to test specifications, designs and other products and documents
- When used to test code, it can be used early to test code that cannot yet be executed
  - Perhaps during dry runs of code incomplete code, pseudocode or flow graphs
- Static test objectives: improving quality, detecting defects, assessing testability, etc.
- Examples of static tests include reviews and static analysis

# Dynamic Testing

- Dynamic testing is class of testing that requires code execution
- Typically works by causing failures from which the associated defects are determined
  - It is perhaps what most people think of as software testing
- Can only be applied to executable test objects
  - However, this can be done earlier than was traditionally the case, as in TDD
- Dynamic testing objective is largely to improve quality through detecting defects
- Example of dynamic tests include, functional and non-functional testing

# Static and Dynamic Testing

- Static testing and dynamic testing have the same aim, to improve software quality
- As such they complement each and are both parts of a well structured test plan
- However, the balance of testing types used will be informed by the overarching SDLC



# Test Techniques and Approaches

# Test Techniques

- There are a number of established classes of test techniques, including:
  - Black-Box techniques
  - White-Box techniques
  - Experience-based techniques
- Whatever the test techniques, they structure what to test and how to test it
- This helps to systematically develop sets of test cases with optimal
  - Test coverage
  - Test data requirements
- With the goal of maximising defect detection as reliably and efficiently as possible
- The broader approach taken to implement test techniques is important
  - Such as taking a collaborative, defect avoidance or focused approach

# Black Box Test Techniques

- Specification-based deriving tests from documentation external to the test object
- Objective is to check the software behaviour against its specifications
- Does not use knowledge of the internal structure or operation of the test object
- Test cases are independent of how the underlying software has been implemented
- So, test cases can remain useful even if implementation of the behaviour has changed
- Equivalence Partitioning,
  - Group test data into partitions, then design test cases at the partition level
  - Assume test object defects will be revealed by any test data within a partition
- Boundary Value Analysis
  - Exercise the boundaries of ordered equivalence partitions
  - Uses minimum and maximum values of a partition as the boundary to be tested



# White Box Test Techniques

- Structure-based deriving tests from the software implementation or internal structure
  - Objective is to provide test overage of the internal structure to an acceptable level
  - Informed by the internal structure and operation of the test object
  - Test cases cannot be designed until the software has been designed or implemented
  - However, can be used in static testing, during dry runs of code, pseudocode or flow graphs
  - Importantly, White-Box testing will not detect omitted requirements
- 
- Statement testing
    - Tests are designed to exercise as many statements in the code as is acceptable
    - However, exercising a statement does not guarantee detection of any defect present
  - Branch testing
    - Tests exercise branches in the code until an acceptable level of coverage is achieved
    - Again, exercising a branch does not guarantee detection of any defect present



# Experience-Based Test Techniques

- This technique uses tester experience of testing and development to design tests
- Can detect defects missed using Black-Box and White-Box techniques
- Highly sensitive to the experience of the tester conducting the testing
- As such, they complement Black-Box and White-Box techniques but do not replace them
- There are a several established techniques that can be used:
  - **Error guessing**, anticipating defects based on the tester's past experiences
  - **Exploratory testing**, learning about the software and how to test it, by testing it
  - **Checklist-based testing**, using questions based on knowledge of software or users



# Collaboration-Based Test Approaches

- An approach rather than a technique
- Part of a whole-team approach
- The whole team is responsible for software quality
- Any team member with the necessary knowledge can perform tests
- Testing is however guided by testers, working with the team and stakeholders
  - Agree a test strategy
  - Help to create suitable tests
  - Select test automation approaches that suit everyone
- As a result, knowledge of testing is transferred to all team members
- This results in testing influencing development at a fundamental level
- Creating a more proactive approach that aims to prevent defects, not just find them
- Which helps to realise higher software quality, sooner, and at lower cost

# Test Levels

# Test Levels

- Test level are groups of test activities related to a given stage in the development process
- A level could relate to a component or a complete system, depending on the SDLC
- In some SDLCs each test level can be seen as a separate instance of the testing process
  - Sequential SDLCs may required exit criteria from one level before starting the next level
  - Although test levels may overlap in time, as exit criteria become available
  - Iterative SDLCs tend to accommodate development activities spanning multiple levels



# Component Testing

- Component testing, or unit testing
- Tests units in isolation
- Might make use of tools to support working within an unfinished system
  - Tools such as, test harnesses or unit test frameworks
- Normally performed by developers in their development environments
  - May be heavily automated within some practices

# Component Integration Testing

- Component integration testing, or simply integration testing
- Tests the interfaces and interactions between components
- Heavily dependent on the integration strategy
  - Such as, bottom-up, top-down or big-bang

# System Testing

- Tests the overall behaviour and capabilities of the software, including:
  - Functional testing of complete tasks
  - Non-functional testing of usability



# System Integration Testing

- Tests the interfaces to other systems and external services
- Normally performed within an operational-like environment

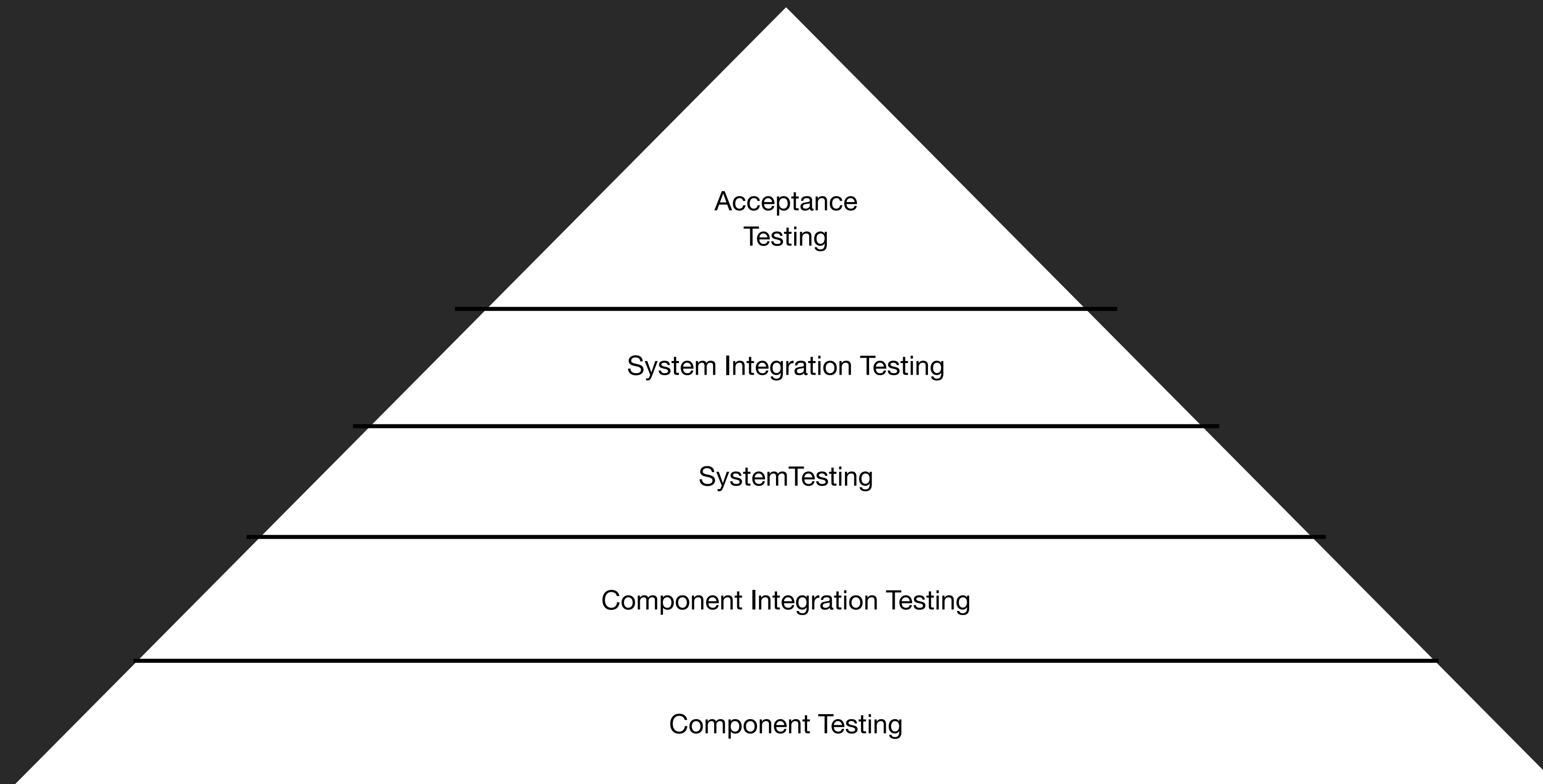
# Acceptance Testing

- Tests that the software meets the user's needs
- Validates the software is ready to be deployed
- Normally involves the users
- Can take a few forms, depending on the focus:
  - User acceptance testing
  - Operational acceptance testing
  - Contractual acceptance testing
  - Regulatory acceptance testing
  - Alpha testing
  - Beta testing



# Testing Pyramid

- The Testing Pyramid is a model showing the different test levels within an approach
  - Intended to model test automation
  - We can adapt a version here to help understand the relationship between test layers



# Test Types

# Test Types

- Test types are groups of test activities organised by the quality characteristic they test
  - Functional Tests
  - Non-Functional Tests
  - Black-Box Tests
  - White-Box Tests
- Test types can be performed at any level, where that quality characteristic needs tested
- *Note, we have already considered the Box models earlier, at a higher level*



# Functional Tests

- Tests the intended function of the test object
- Tests “What” the test object should do
- Objective is to check aspects such as:
  - Functional completeness
  - Functional correctness
  - Functional appropriateness

# Non-functional Tests

- Tests attributes that are not functional in nature
- Testing “how” the software behaves
- Objective is to test characteristics including:
  - Performance efficiency
  - Compatibility
  - Usability
  - Reliability
  - Security
  - Maintainability

# Non-functional Test Considerations

- Non-functional tests often require completed components to allow testing
- It can be started early, using review or during component testing or system testing
- Can perhaps reuse functional tests but with the focus on non-functional outcomes
- This can change the nature of the functional test, and the required tester skill set
- However, late discovery of non-functional defects can present significant risks
- As such, it can be worth the complexity and cost that doing so introduces



# Testing Change

# Change

- Change is a characteristic of most software systems both during and after development
- Changes can be necessary during development, in response to:
  - Testing, e.g. component, integration or acceptance testing
  - Evolving understanding of the problems and its solution
- Changes can also be required following release, in response to:
  - Evolving stakeholder needs
  - Changes to the operational environment
- Changes should be tested using Confirmation, Regression and Maintenance testing



# Confirmation Testing

- Confirmation testing checks that changes due to defects have been made successfully
- The objective is to confirm that the defect being addressed has been fixed
- The nature of the tests can be guided by the level of risk the defect presented
  - For low risks, perhaps relying on the tests that failed due to the defect
  - For more significant risks, creating new tests to cover the change
- The former is risky but can be quicker and cheaper, depending on long-term outcome

# Regression Testing

- Regression testing checks for adverse consequences resulting from change
  - Perhaps following the need for confirmation testing
  - However, regression testing is a very standard practice during ongoing development
- Following change, adverse consequences may be present in:
  - The component where the change was made
  - Other components in the same system
  - Connected systems, and the connection to them
- Objective is to ensure the software has not been adversely effected by the change

# Regression Testing Impact Analysis

- Regression testing may need to extend beyond the test object
- In some cases an impact analysis might be appropriate
- This helps to determine the extent and nature of the regression testing required
- It does so by identifying which parts of the software could be affected



# Regression Testing Automation

- Regression testing can be a frequent activity during development
  - The extent of regression testing increases as the software becomes more complex
  - This can result in a significant workload
- The nature of many regression tests are well suited to being automated
- Test automation is of most benefit when introduced early in a project
  - This is the case for automated testing in general, not just automating regression testing

# Maintenance Testing

- Software maintenance can be planned or unplanned, and categorised as:
  - Corrective, in response to defects
  - Adaptive, necessitated by changes in the operational environment
  - Improvements, in performance or maintainability
- Triggers for software maintenance, and associated testing can be classified as:
  - Modifications, planned upgrades, released, corrective changes or hot fixes
  - Upgrades/migrations of operational environment, e.g. platform upgrade or migration
  - Retirement of the software, e.g. end of its life or data archiving

# Impact Analysis

- Testing changes to a system in production includes:
  - Evaluating the success of the implementation of the change, Confirmation testing
  - Checking for possible regressions throughout the system, Regression testing
- The scope of testing depends on:
  - The extent of the change
  - The significant of the risk present by the change
  - The size of the software system
- Impact analysis can help to decide if a change should be made
  - This will help to understand the potential consequences to the software



# Test Planning

# Test Plan

- Testing is a complex set of activities that are critical to the success of a project
- Careful planning is important to ensure its effectiveness
- Preparing a test plan helps to consider what is needed to achieve the test objectives
- It encourages forward thinking regarding test activities, uncovering challenges relating to:
  - Risks
  - Schedules
  - Stakeholders
  - Tools
  - Costs
  - Effort



# Test Plan Purpose

- Purpose of a test plan is to detail how test objectives will be achieved reliably
- The test plan sets the test objectives
- Defines how the test objectives will be achieved
- Schedules activities for achieving the test objectives
- Describes how test activities will meet the established criteria
- Makes clear the nature of communications within the team and with stakeholders
- States how testing will follow a test policy or test strategy

# Test Plan Content

- The exact nature of any test plan will depend on many factors
- However, there are a range of key areas, each with some common content to consider
- **Context of testing**, scope, test objectives, constraints
- **Assumptions and constraints**, at a project level
- **Stakeholders**, roles, responsibilities, relevance to testing, training needs
- **Communication**, form, frequency, format
- **Risks**, risk register relating to both product risks and project risks
- **Test approach**, test levels, test types, test techniques, test deliverables, entry criteria, exit criteria, test independence, metrics, test data requirements, test environment requirements, test strategy
- **Schedule**, test activity sequence, interdependence
- **Budget**, the costs associated with the planned testing

# Relationship between Test Planning and Project Planning

- In iterative SDLCs test planning has a relationship with project level planning
- Release Planning:
  - Writing testable user stories and acceptance criteria
  - Project and product risks
  - Effort required to test user stories
- Iteration planning:
  - Detailed risk analysis of user stories
  - Testability of user stories
  - Breaking down user stories into tasks
  - Estimate test effort for tasks
  - Consideration of functional and non-functional test object



# Entry Criteria

- Entry criteria are the preconditions for an activity
- They should be defined for each test level based on the test objectives
- Typical entry criteria include:
  - Resources availability, people, tools, environments, test data, budget, time
  - Testable requirements or user stories are available
  - Test cases are availability
  - Preparatory quality assessment of test objects has been passed
- If entry criteria are not met then the activity is less likely to be successful
  - Perhaps involving increased difficulty, time, cost, and risk
- In Agile, Definition of Ready can serve as entry criteria
  - The criteria a user story must meet to start development and testing

# Exit Criteria

- Exit criteria define what must be achieved in order to declare an activity completed
- Exit criteria should be defined for each test level based on the test objectives
- Typical exit criteria include:
  - Thoroughness metrics, test coverage, defect number, density, state, and failed test
  - Completion criteria, tests executed, static tests performed, defects reported
  - Running out of time or budget can be considered as valid exit criteria, and testing exited
  - Stakeholders have accepted test object and the risk of release without further testing
- In Agile, Definition of Done can serve as exit criteria
  - The criteria defining the objective metrics for a releasable item

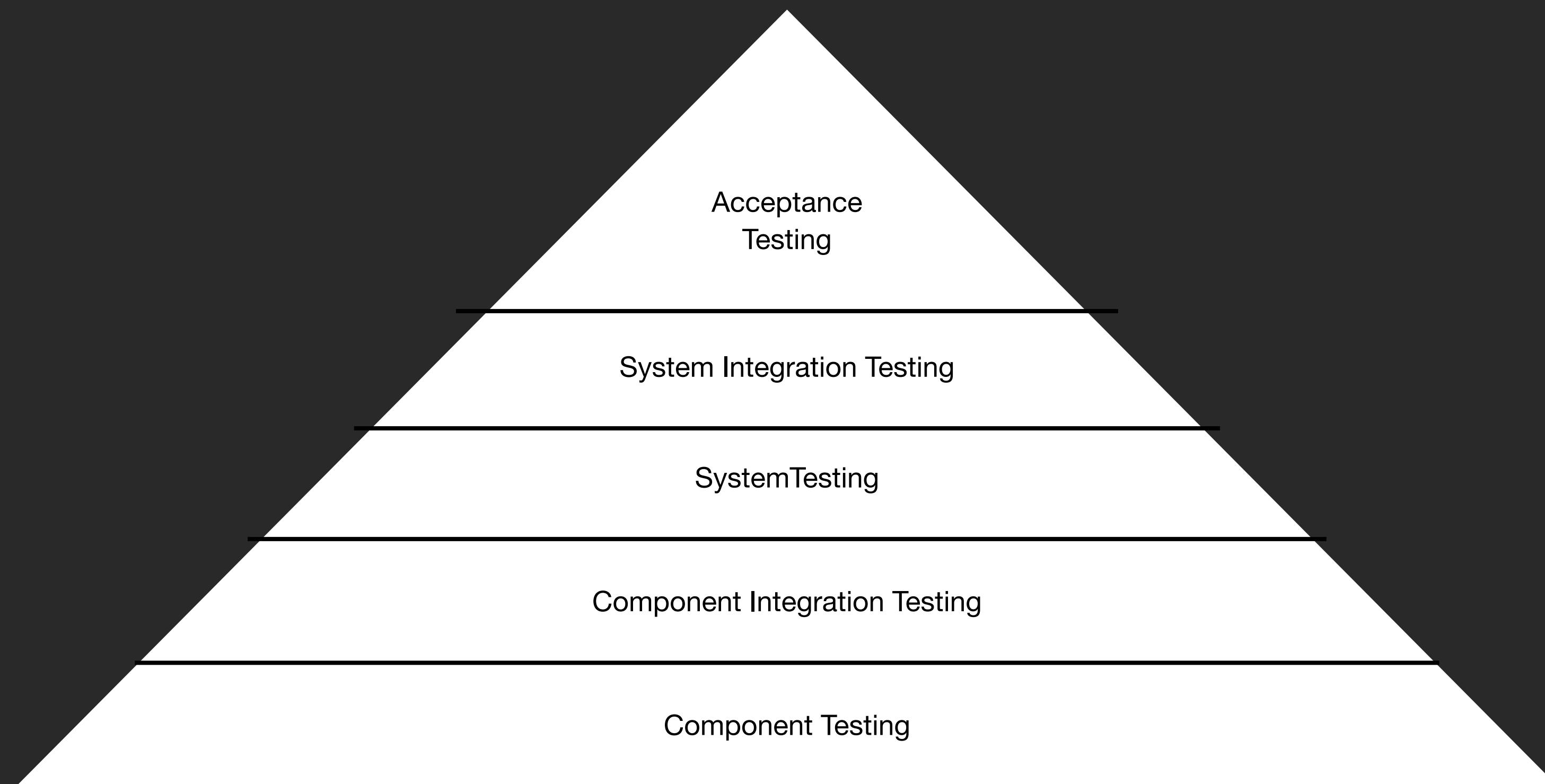
# Test Case Prioritisation

- To allow scheduling, tests must be prioritised to determine the order of execution
- Common test case prioritisation strategies are:
  - Risk-based, order based on the results from risk analysis, to address highest risks first
  - Coverage-based, order based on the degree of coverage offered by test cases
  - Requirements-based, order based on the priorities of the corresponding requirements
- There are additional factors that prioritisation must take into account
  - Dependencies between test objects can require lower priority to be executed early
  - Resource availability, e.g. people may dictate when a test case can be executed



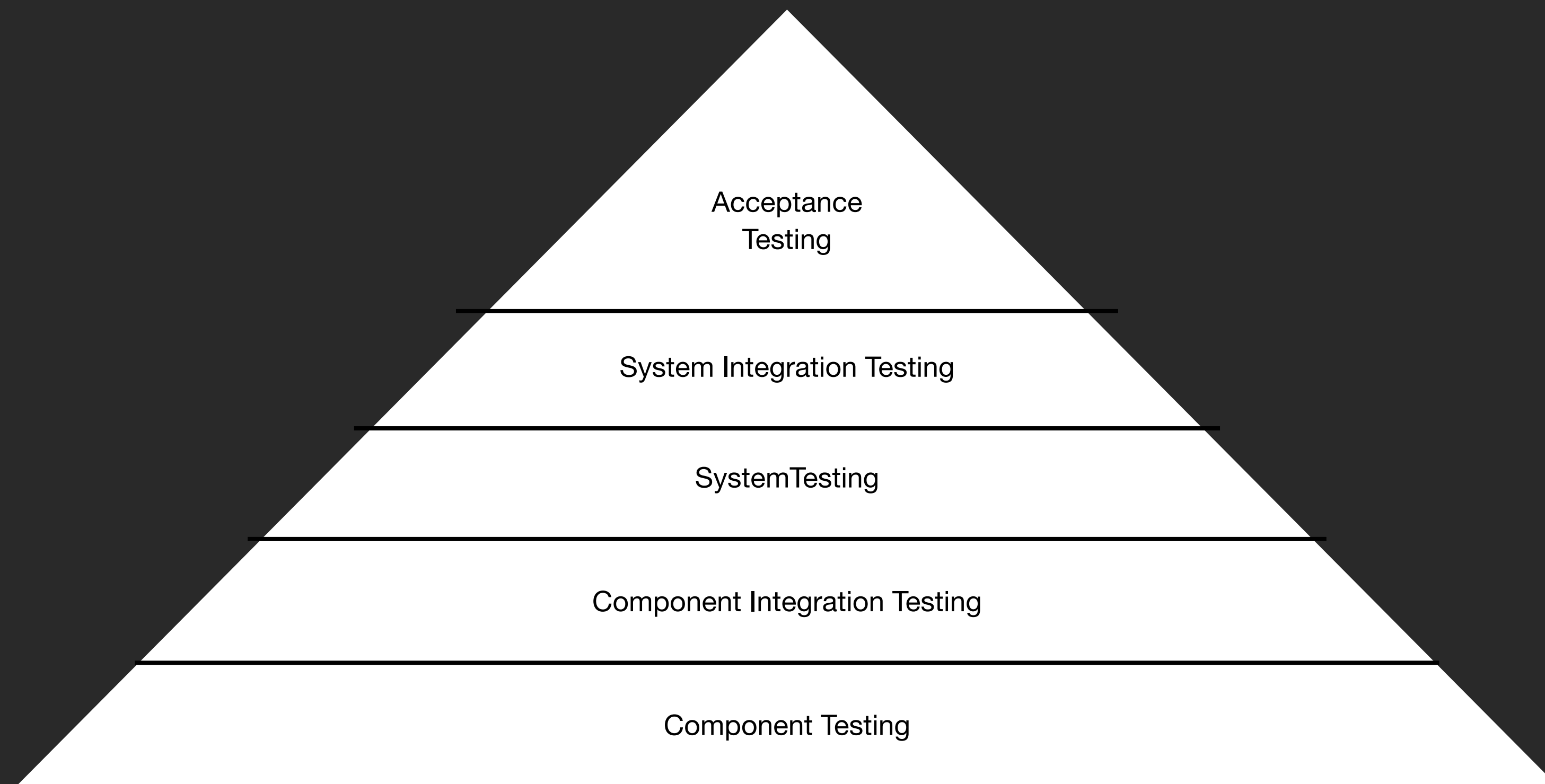
# Test Pyramid

- The Pyramid model shows how different types of tests have different granularity
- As a general rule, pushing tests as low as possible increases automation
- Reducing time and cost and making further automation and savings possible



# Test Pyramid Structure

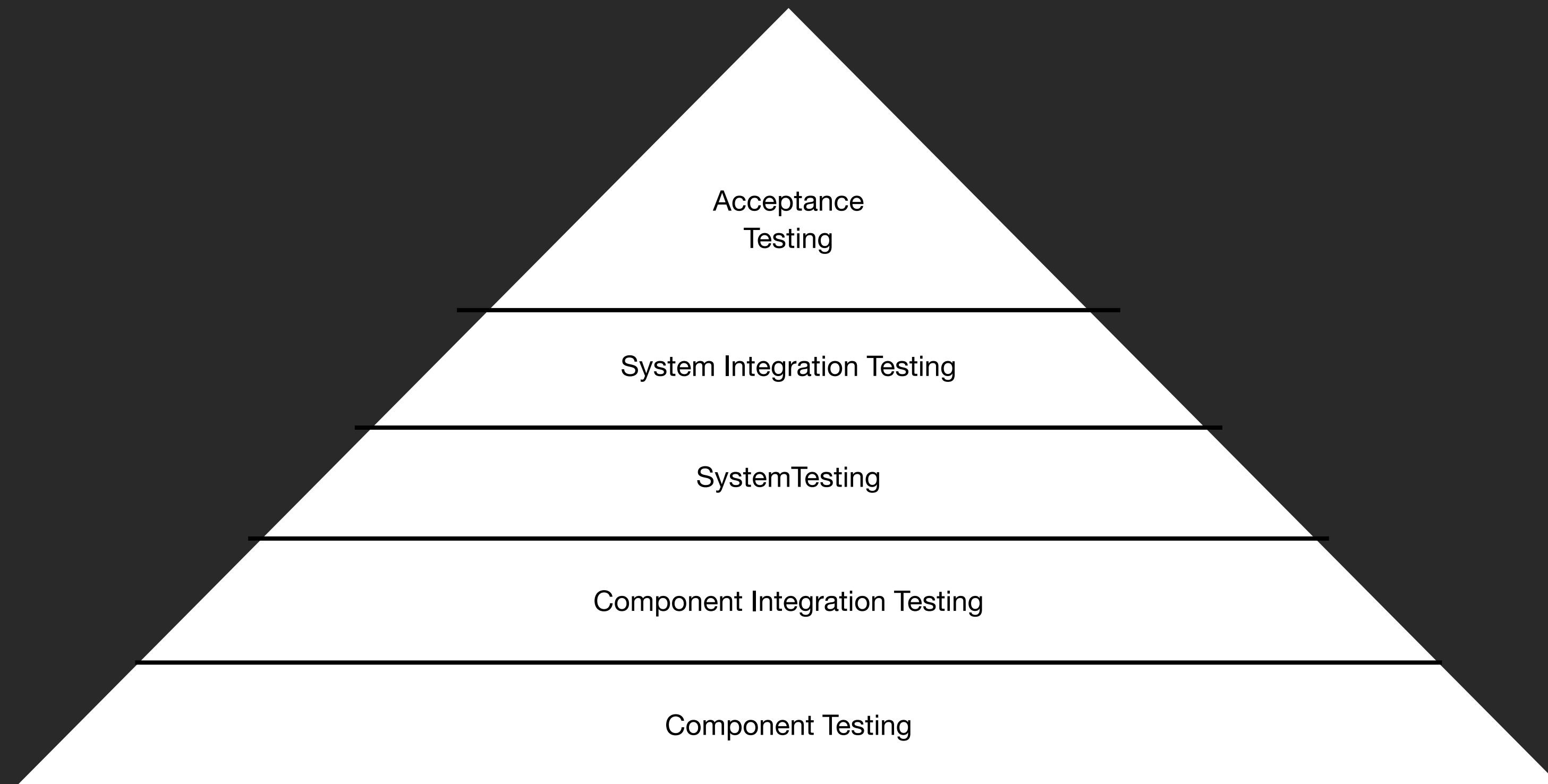
- Pyramid structure
  - Layers represent groups of tests
  - Higher layer tests are coarser: larger, less isolated, slower, and have broader coverage
  - Lower layer test are finer: smaller, more isolated, faster, and have narrower coverage





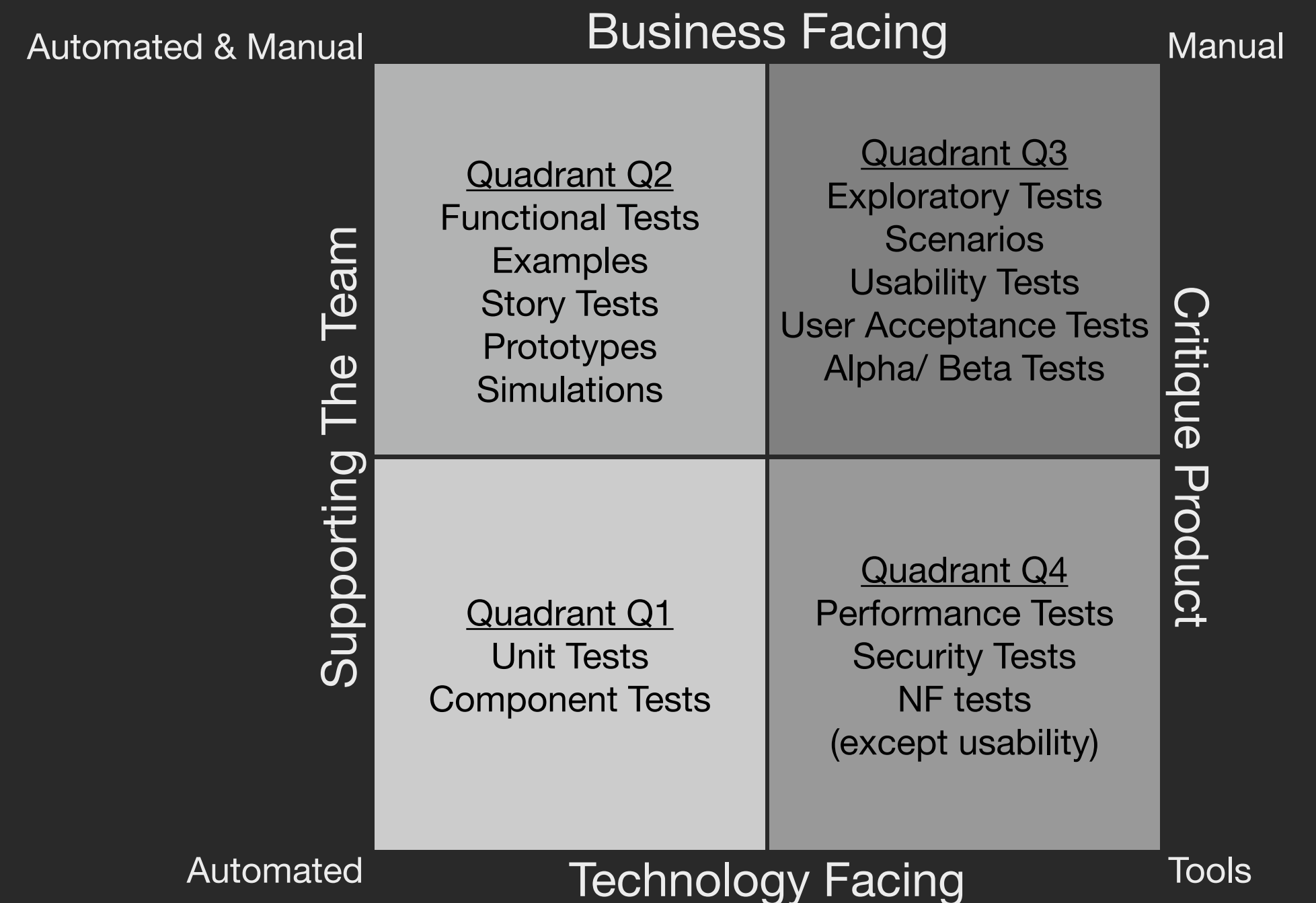
# Test Pyramid Variations

- The number and naming of the layers may differ between versions
- The original model defines three layers: Unit tests, Service tests and UI tests
- Another popular version defines Unit tests, Integration tests, and End-to-end tests
- There are others versions to suit different approaches to testing



# Testing Quadrants

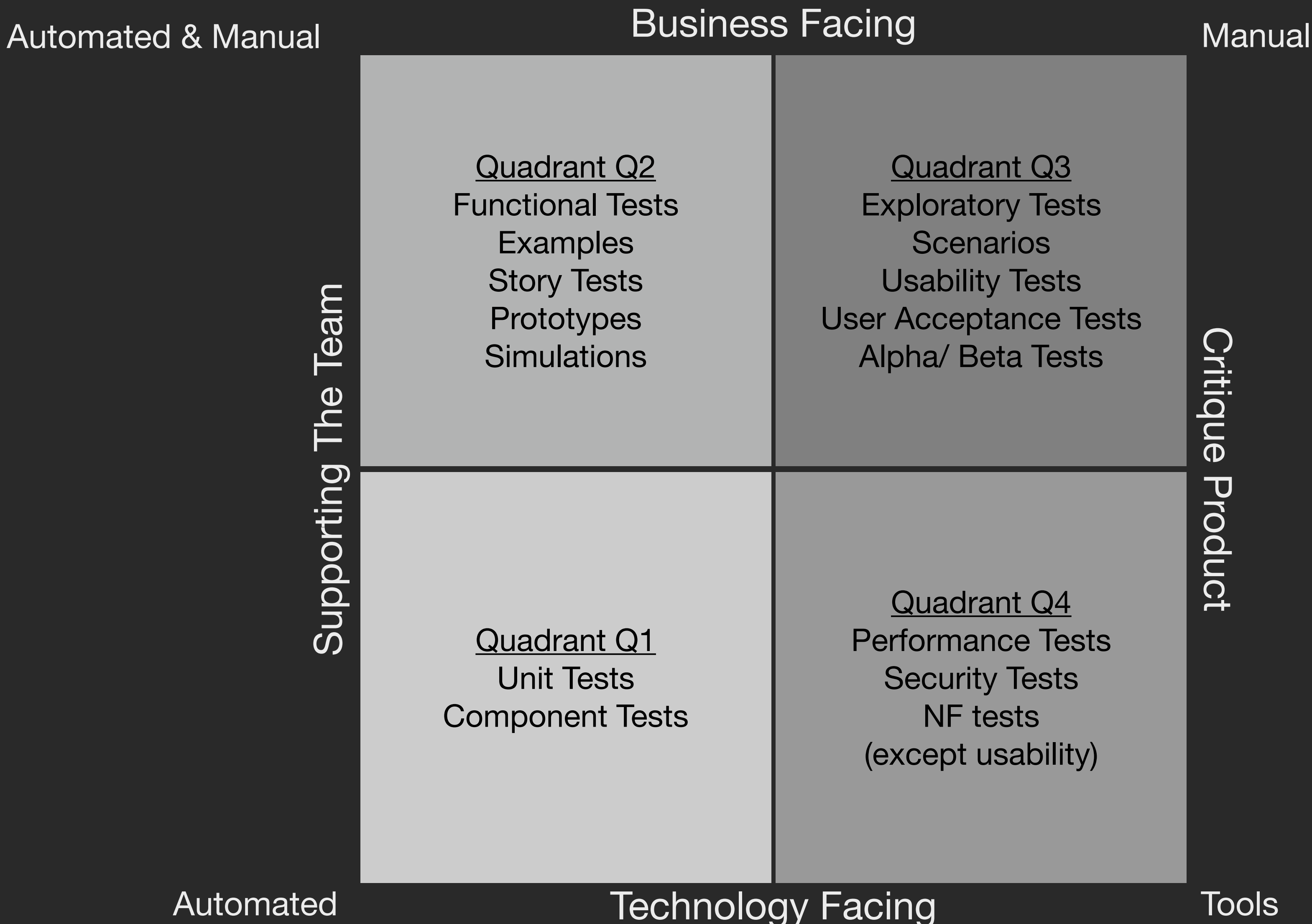
- The Testing Quadrants model groups and visualises:
  - Test levels
  - Test types
  - Activities
  - Test techniques
  - Agile work products
- Items are grouped by to main factors
  - Business facing or technology facing
  - Supporting team (guide development) or critiquing product (measuring product behaviour)
- These factors can be modelled using four resultant quadrants:



# Testing Quadrants

## Quadrant Q1

- Technology facing
- Supports the team
- Guides development
- Check function
- Should be automated

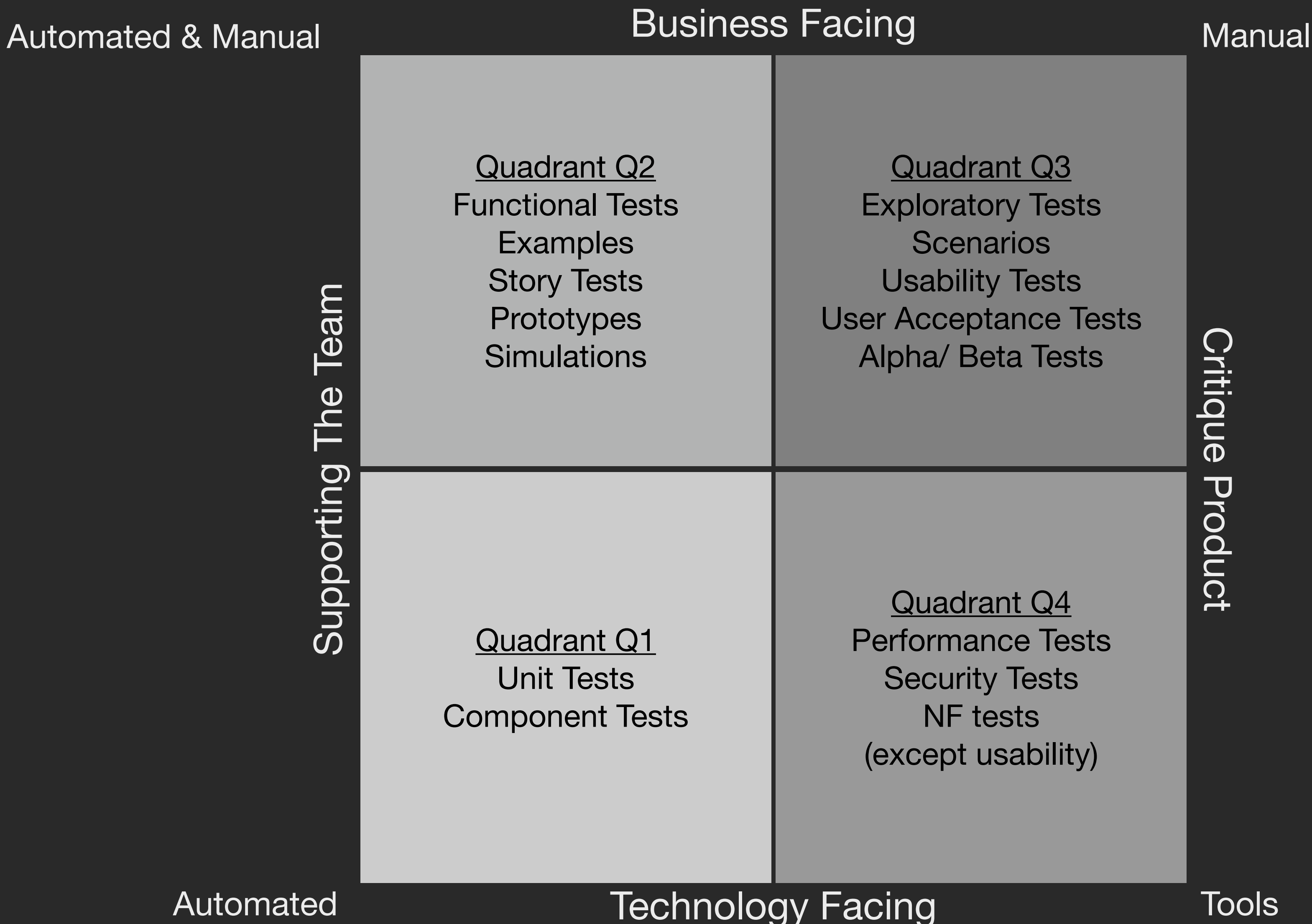




# Testing Quadrants

## Quadrant Q2

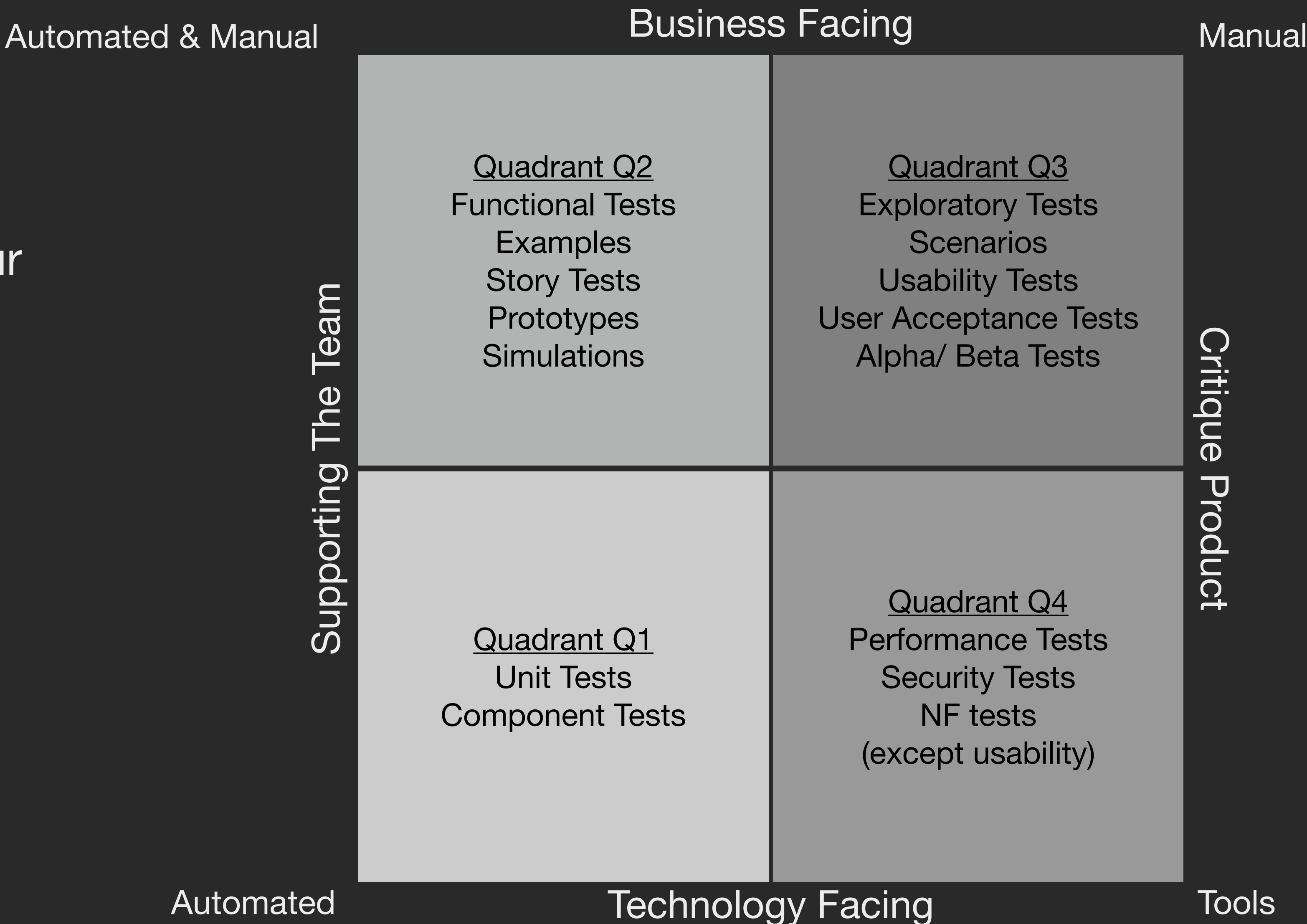
- Business facing
- Supports the team
- Guides Development
- Checks acceptance criteria
- Manual or automated



# Testing Quadrants

## Quadrant Q3

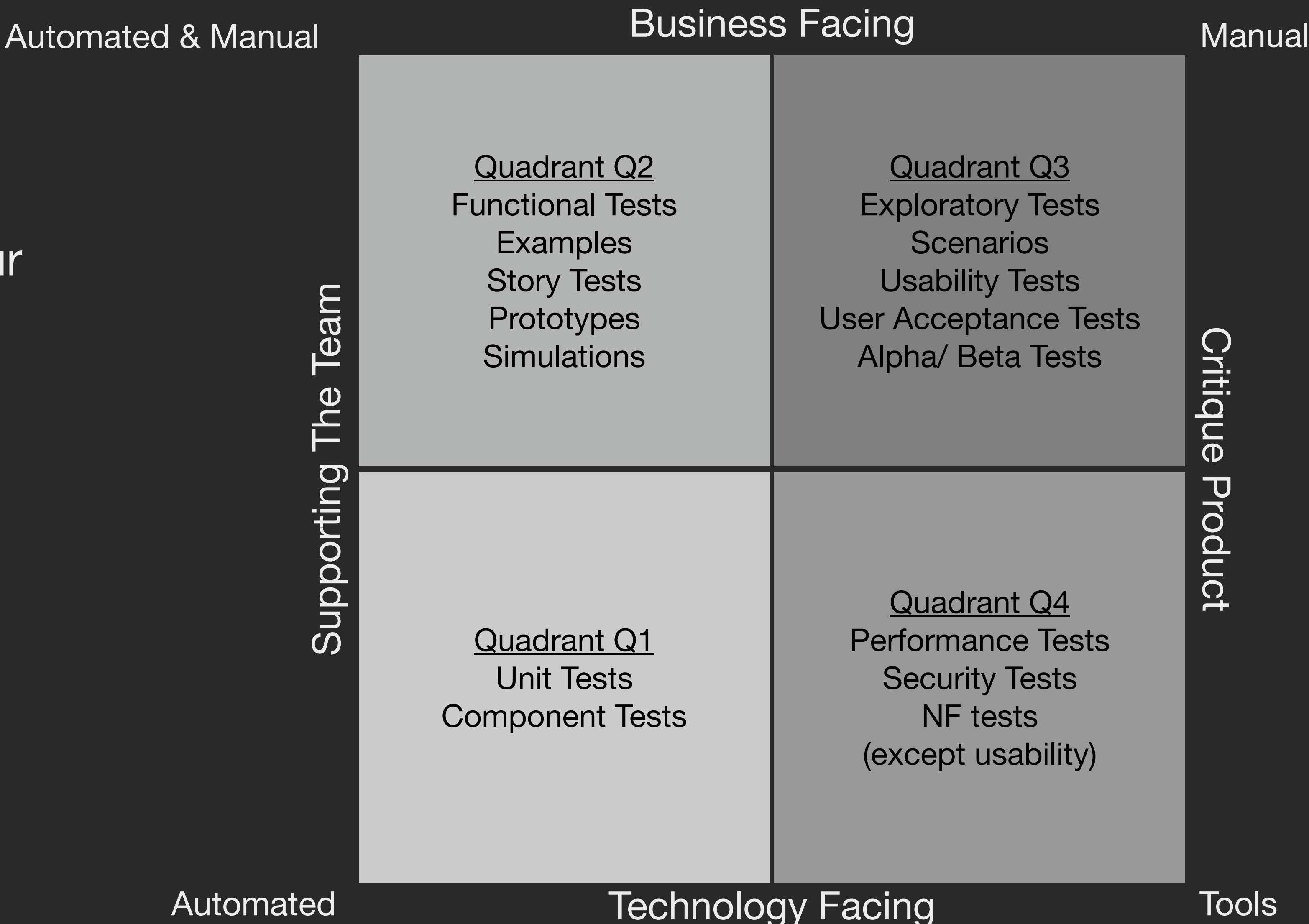
- Business facing
- Critique the product
- Measures product behaviour
- Checks product
- User-oriented
- Often manual



# Testing Quadrants

## Quadrant Q4

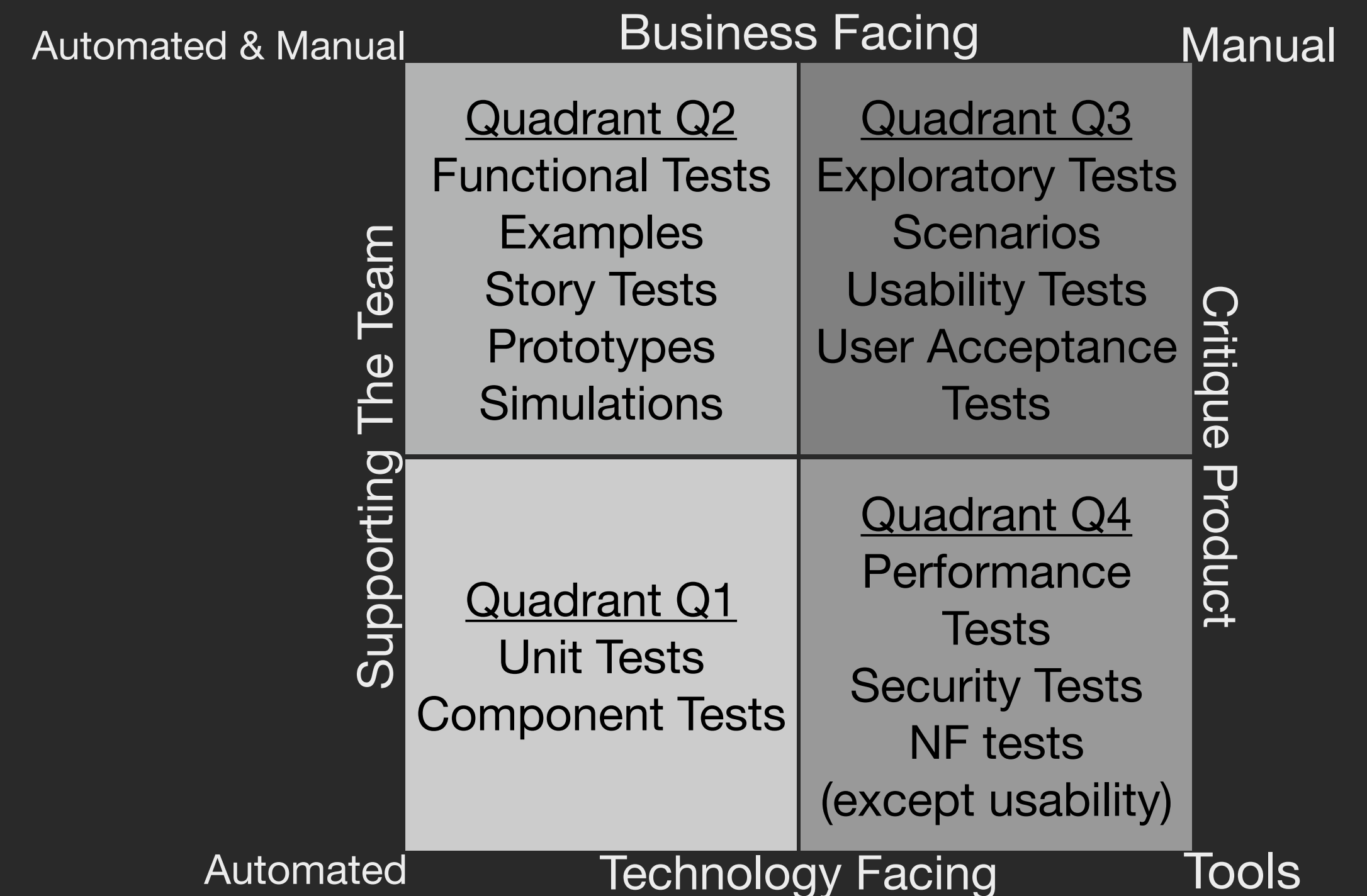
- Technology facing
- Critique the product
- Measures product behaviour
- Checks operation
- Often automated





# Testing Quadrants Value

- It supports test management by mapping these items against key factors
- Helps to ensure all appropriate test types and test levels are included in the SDLC
- Visualises that some test types are more relevant to certain test levels than others
- It is assessable to most stakeholders when describing test types



# Test Tools



# Test Tools

- Test tools can be used to support
  - Test management
  - Static testing
  - Test design
  - Test case generation
  - Test data generation
  - Test execution
  - Coverage analysis
  - Non-functional testing
  - Communication

# Benefits of Test Automation

- Reduced test execution time
- Increased frequency at reduced time cost
- Increased consistency and repeatability while reducing time and effort
- More objective and measurable assessment
- Richer logging for analysis
- Earlier defect detection

# Risks of Test Automation

- Unrealistic expectations
- Cost in time to adapt automation
- Using automated testing inappropriately, instead of manual testing
- Using tools that are not accessible to stakeholders required to support testing
- Incompatibilities between tools
- Tool vendor lock-in
- Ignoring human critical thinking



# Defect Management

# Defect Management

- A major goal of testing is defect detection
- A robust process is required to manage defect processing
- Defect management process must handle individual anomalies from discovery to closure
- Defect management process typically includes:
  - Defect logging
  - Defect analysis
  - Defect classification
  - Response selection
  - Closing a defect

# Defect Report

- Defect report makes it the easier to replicate and address the defect in a controlled way
- Usefully, automated testing can automatically generate much of the required details
- A typical dynamic testing defect report might include:
  - Unique identifier, defect title, and Description of the failure
  - Date observed and by who
  - Identification of the test object
  - Context of the defect and the steps to retreat it
  - Related test case with expected and actual results
  - Test logs, database dumps, or screenshots
  - Severity of the defect and the priority of a fix
  - Status, e.g. open, deferred fixed, closed



# CW Test Case Example

# CW Test Case Example

| Test Case ID: TC101                                                                       |                                                                                                        |            |                                                                          |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------|--------------------------------------------------------------------------|
| Test Description: Login to a user’s dashboard using a valid username and a valid password |                                                                                                        |            |                                                                          |
| User Story ID: S18                                                                        | User Story Title: As a customer I want to be able to manage the date that the website stores about me. |            |                                                                          |
| Step ID                                                                                   | Step Description                                                                                       | Test Data  | Expected Result                                                          |
| 1                                                                                         | Browse to login page                                                                                   |            | Log in page is displayed and username form field has the focus.          |
| 2                                                                                         | Enter valid username                                                                                   | validuser  | Username is accepted and password field has the focus.                   |
| 3                                                                                         | Enter valid password                                                                                   | vu19qwerty | Password is accepted and form focus moves to log in button.              |
| 4                                                                                         | Click “Log In” button                                                                                  |            | Log in is successful and the related user’s dashboard page is displayed. |



# CW Test Case Example

| Test Case ID: TC102                                                                          |                                                                                                        |             |                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test Description: Login to a user’s dashboard using an invalid username and a valid password |                                                                                                        |             |                                                                                                                                                                              |
| User Story ID: S18                                                                           | User Story Title: As a customer I want to be able to manage the date that the website stores about me. |             |                                                                                                                                                                              |
| Step ID                                                                                      | Step Description                                                                                       | Test Data   | Expected Result                                                                                                                                                              |
| 1                                                                                            | Browse to login page                                                                                   |             | Log in page is displayed and username form field has the focus.                                                                                                              |
| 2                                                                                            | Enter invalid username                                                                                 | invaliduser | Username is accepted and password field has the focus.                                                                                                                       |
| 3                                                                                            | Enter valid password                                                                                   | iu20qwerty  | Password is accepted and form focus moves to log in button.                                                                                                                  |
| 4                                                                                            | Click “Log In” button                                                                                  |             | Login is unsuccessful, a login error message is displayed instructing that the username or password are invalid and asking them to try again. The login field has the focus. |

# Reading

- This lecture is informed by, and structured to match, ISTQB certification guidance
  - [ISTQB® Certified Tester Foundation Level, 2023](#)
  - [ISTQB® Certified Tester Foundation Level - Agile Tester, 2014](#)
- It is also informed by the following books, which make good further reading
  - [Agile Testing, Lisa Crispin and Janet Gregory, 2008](#)
  - [Software Testing, edited by Brian Hambling. 2019](#)
  - [Agile Testing Foundations, edited by Rex Black, 2017](#)

*Note, book links above are to O'Reilly Books Online, for which Ulster University students have full text access.*

# Testing